Nama: Hans Malvin Djojo

https://github.com/hansmalvin/LastMile

Step 1: Design the Embedded Data Model

```
{
  "_id": ObjectId(),
  "name": "Alice Smith",
  "email": "alice@example.com",
  "orders": [
    {
      "orderId": ObjectId(),
      "orderDate": ISODate("2023-10-26T10:00:00Z"),
      "totalAmount": 120.00,
      "items": [
        {
          "productId": ObjectId(),
          "productName": "Laptop",
          "quantity": 1,
          "price": 120.00
        }
      ]
    }
  ]
}
```

Step 2: Insert Data Using the Embedded Model

```
ecommerce> db.customers.insertOne({
|   name: "Bob Johnson",
|   email: "bob@example.com",
|   orders: [
|     {
|       orderId: new ObjectId(),
|       orderDate: new Date("2023-11-15T14:30:00Z"),
|       totalAmount: 75.50,
|       items: [
|         { productId: new ObjectId(), productName: "Mouse", quantity: 1, price: 25.00 },
|         { productId: new ObjectId(), productName: "Keyboard", quantity: 1, price: 50.50 }
|       ]
|     },
|     {
|       orderId: new ObjectId(),
|       orderDate: new Date("2023-11-20T09:00:00Z"),
|       totalAmount: 30.00,
|       items: [
|         { productId: new ObjectId(), productName: "USB Cable", quantity: 2, price: 15.00 }
|       ]
|     }
|   ]
| });
{
  acknowledged: true,
  insertedId: ObjectId('69ef8c80a4492d6e513682d6')
}
```

```
ecommerce> db.customers.find({ name: "Bob Johnson" })
[
  {
    _id: ObjectId('69ef8c80a4492d6e513682d6'),
    name: 'Bob Johnson',
    email: 'bob@example.com',
    orders: [
      {
        orderId: ObjectId('69ef8c80a4492d6e513682d1'),
        orderDate: ISODate('2023-11-15T14:30:00.000Z'),
        totalAmount: 75.5,
        items: [
          {
            productId: ObjectId('69ef8c80a4492d6e513682d2'),
            productName: 'Mouse',
            quantity: 1,
            price: 25
          },
          {
            productId: ObjectId('69ef8c80a4492d6e513682d3'),
            productName: 'Keyboard',
            quantity: 1,
            price: 50.5
          }
        ]
      },
      {
        orderId: ObjectId('69ef8c80a4492d6e513682d4'),
        orderDate: ISODate('2023-11-20T09:00:00.000Z'),
        totalAmount: 30,
        items: [
          {
            productId: ObjectId('69ef8c80a4492d6e513682d5'),
            productName: 'USB Cable',
            quantity: 2,
            price: 15
          }
        ]
      }
    ]
  }
]
```

Step 3: Design the Referenced Data Model

```
{
_id: ObjectId(),
name: "Charlie Brown",
email: "charlie@example.com"
}

{
_id: ObjectId(),
customerId: ObjectId("..."),
orderDate: ISODate("2023-11-22T12:00:00Z"),
```

totalAmount: 90.00,
items: [
{
productId: ObjectId(),
productName: "Headphones",
quantity: 1,
price: 90.00
}
]
}

Step 4: Insert Data Using the Referenced Model

```
ecommerce> db.customers.insertOne({
| name: "David Lee",
| email: "david@example.com"
| })
{
  acknowledged: true,
  insertedId: ObjectId('69ef8d92a4492d6e513682d7')
}
ecommerce> let customer = db.customers.findOne({ name: "David Lee" });
| let customerId = customer._id;
| print(customerId);
ObjectId('69ef8d92a4492d6e513682d7')

ecommerce> db.orders.insertOne({
| customerId: customerId,
| orderDate: new Date("2023-11-25T16:00:00Z"),
| totalAmount: 55.00,
| items: [
| { productId: new ObjectId(), productName: "Webcam", quantity: 1, price: 55.00 }
| ]
| })
| db.orders.insertOne({
| customerId: customerId,
| orderDate: new Date("2023-11-28T11:00:00Z"),
| totalAmount: 110.00,
| items: [
| { productId: new ObjectId(), productName: "Monitor", quantity: 1, price: 110.00 }
| ]
| })
{
  acknowledged: true,
  insertedId: ObjectId('69ef8dbca4492d6e513682db')
}
ecommerce>
```

```
ecommerce> db.customers.find({ name: "David Lee" })
[
  {
    _id: ObjectId('69ef8d92a4492d6e513682d7'),
    name: 'David Lee',
    email: 'david@example.com'
  }
]
ecommerce> db.orders.find({ customerId: customerId })
[
  {
    _id: ObjectId('69ef8dbca4492d6e513682d9'),
    customerId: ObjectId('69ef8d92a4492d6e513682d7'),
    orderDate: ISODate('2023-11-25T16:00:00.000Z'),
    totalAmount: 55,
    items: [
      {
        productId: ObjectId('69ef8dbca4492d6e513682d8'),
        productName: 'Webcam',
        quantity: 1,
        price: 55
      }
    ]
  },
  {
    _id: ObjectId('69ef8dbca4492d6e513682db'),
    customerId: ObjectId('69ef8d92a4492d6e513682d7'),
    orderDate: ISODate('2023-11-28T11:00:00.000Z'),
    totalAmount: 110,
    items: [
      {
        productId: ObjectId('69ef8dbca4492d6e513682da'),
        productName: 'Monitor',
        quantity: 1,
        price: 110
      }
    ]
  }
]
ecommerce>
```

Step 5: Querying Data (Embedded Model)

```
ecommerce> db.customers.find({ name: "Bob Johnson" })
[
  {
    _id: ObjectId('69ef8c80a4492d6e513682d6'),
    name: 'Bob Johnson',
    email: 'bob@example.com',
    orders: [
      {
        orderId: ObjectId('69ef8c80a4492d6e513682d1'),
        orderDate: ISODate('2023-11-15T14:30:00.000Z'),
        totalAmount: 75.5,
        items: [
          {
            productId: ObjectId('69ef8c80a4492d6e513682d2'),
            productName: 'Mouse',
            quantity: 1,
            price: 25
          },
          {
            productId: ObjectId('69ef8c80a4492d6e513682d3'),
            productName: 'Keyboard',
            quantity: 1,
            price: 50.5
          }
        ]
      },
      {
        orderId: ObjectId('69ef8c80a4492d6e513682d4'),
        orderDate: ISODate('2023-11-20T09:00:00.000Z'),
        totalAmount: 30,
        items: [
          {
            productId: ObjectId('69ef8c80a4492d6e513682d5'),
            productName: 'USB Cable',
            quantity: 2,
            price: 15
          }
        ]
      }
    ]
  }
]
ecommerce>
```

```
ecommerce> db.customers.aggregate([
| {
| $match: {
| "orders.orderDate": { $gt: new Date("2023-11-16T00:00:00Z") }
| }
| }
| ])
[
  {
    _id: ObjectId('69ef8c80a4492d6e513682d6'),
    name: 'Bob Johnson',
    email: 'bob@example.com',
    orders: [
      {
        orderId: ObjectId('69ef8c80a4492d6e513682d1'),
        orderDate: ISODate('2023-11-15T14:30:00.000Z'),
        totalAmount: 75.5,
        items: [
          {
            productId: ObjectId('69ef8c80a4492d6e513682d2'),
            productName: 'Mouse',
            quantity: 1,
            price: 25
          },
          {
            productId: ObjectId('69ef8c80a4492d6e513682d3'),
            productName: 'Keyboard',
            quantity: 1,
            price: 50.5
          }
        ]
      },
      {
        orderId: ObjectId('69ef8c80a4492d6e513682d4'),
        orderDate: ISODate('2023-11-20T09:00:00.000Z'),
        totalAmount: 30,
        items: [
          {
            productId: ObjectId('69ef8c80a4492d6e513682d5'),
            productName: 'USB Cable',
            quantity: 2,
            price: 15
          }
        ]
      }
    ]
  }
]
ecommerce>
```

Step 6: Querying Data (Referenced Model)

```
ecommerce> db.customers.find({ name: "David Lee" })
[
  {
    _id: ObjectId('69ef8d92a4492d6e513682d7'),
    name: 'David Lee',
    email: 'david@example.com'
  }
]
ecommerce> db.orders.find({ customerId: customerId })
[
  {
    _id: ObjectId('69ef8dbca4492d6e513682d9'),
    customerId: ObjectId('69ef8d92a4492d6e513682d7'),
    orderDate: ISODate('2023-11-25T16:00:00.000Z'),
    totalAmount: 55,
    items: [
      {
        productId: ObjectId('69ef8dbca4492d6e513682d8'),
        productName: 'Webcam',
        quantity: 1,
        price: 55
      }
    ]
  },
  {
    _id: ObjectId('69ef8dbca4492d6e513682db'),
    customerId: ObjectId('69ef8d92a4492d6e513682d7'),
    orderDate: ISODate('2023-11-28T11:00:00.000Z'),
    totalAmount: 110,
    items: [
      {
        productId: ObjectId('69ef8dbca4492d6e513682da'),
        productName: 'Monitor',
        quantity: 1,
        price: 110
      }
    ]
  }
]
ecommerce>
```

```
ecommerce> db.customers.aggregate([
| {
| $match: { name: "David Lee" }
| },
| {
| $lookup: {
| from: "orders",
| localField: "_id",
| foreignField: "customerId",
| as: "orders"
| }
| }
| ])
[
  {
    _id: ObjectId('69ef8d92a4492d6e513682d7'),
    name: 'David Lee',
    email: 'david@example.com',
    orders: [
      {
        _id: ObjectId('69ef8dbca4492d6e513682d9'),
        customerId: ObjectId('69ef8d92a4492d6e513682d7'),
        orderDate: ISODate('2023-11-25T16:00:00.000Z'),
        totalAmount: 55,
        items: [
          {
            productId: ObjectId('69ef8dbca4492d6e513682d8'),
            productName: 'Webcam',
            quantity: 1,
            price: 55
          }
        ]
      },
      {
        _id: ObjectId('69ef8dbca4492d6e513682db'),
        customerId: ObjectId('69ef8d92a4492d6e513682d7'),
        orderDate: ISODate('2023-11-28T11:00:00.000Z'),
        totalAmount: 110,
        items: [
          {
            productId: ObjectId('69ef8dbca4492d6e513682da'),
            productName: 'Monitor',
            quantity: 1,
            price: 110
          }
        ]
      }
    ]
  }
]
```

Step 7: Update Operations

```
ecommerce> db.customers.updateOne(
| { "name": "Bob Johnson", "orders.orderId": ObjectId("69ef8dbca4492d6e513682da") },{ $set: { "orders.$.totalAmount": 80.00 } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 0
}
ecommerce>
```

```
ecommerce> db.orders.updateOne(
| { _id: ObjectId("69ef8dbca4492d6e513682da") }, // Replace ObjectId with the actual orderId
| { $set: { totalAmount: 60.00 } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 0
}
ecommerce>
```

Step 8: Deletion Operations

```
ecommerce> db.customers.updateOne(
| { name: "Bob Johnson" },
| { $pull: { orders: { orderId: ObjectId("69ef8dbca4492d6e513682da") } } } // Replace ObjectId with the actual orderId
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}
ecommerce> db.orders.deleteOne({ _id: ObjectId("69ef8dbca4492d6e513682da") })
{ acknowledged: true, deletedCount: 0 }
ecommerce>
```

Step 9: Observations and Analysis

Observations and Analysis

This lab evaluated the trade-offs between embedded and referenced MongoDB data models using an e-commerce case study. The embedded approach consolidates related information—such as a customer profile and their order history—into a single document. This design excels in read-heavy scenarios, as it retrieves comprehensive data in one fast, join-free query, significantly reducing backend complexity.

However, embedding introduces notable drawbacks. As nested data accumulates, document sizes can balloon, risking performance degradation and potentially hitting MongoDB's strict document size limits. Furthermore, modifying deeply nested fields is often inefficient, and data redundancy can create consistency vulnerabilities if updates are not handled carefully.

Conversely, the referenced model divides data into distinct collections (e.g., separating customers from orders). While this requires aggregation tools like $lookup to join data, it significantly enhances data normalization and scalability. It is particularly well-suited for applications with complex relationships or volatile data, as updating isolated documents is simpler and safer. Ultimately, while the embedded model prioritizes fast read times, the referenced structure offers superior long-term maintainability. The optimal choice relies

heavily on the specific use case, factoring in data volume, update frequency, and query complexity.

Selection Justification

For an enterprise-level e-commerce platform, the referenced data model is the superior choice. Large-scale platforms must manage massive transaction volumes, frequent dynamic updates, and complex entity relationships. Normalizing data into separate collections reduces redundancy and safeguards overall data integrity.

Furthermore, targeted write operations are much more efficient when you only need to modify a specific document rather than rewriting a massive, embedded structure. While retrieving combined data requires relational joins (using $lookup), the performance impact can be effectively managed with strategic database indexing. Ultimately, the referenced model delivers the critical scalability, consistency, and flexibility required to sustain a growing, modern application.